

Deployment — infra/aws/ecs-fargate/ (Terraform)

A from-zero tutorial on Terraform/HCL and the AWS building blocks this module uses, why this deployment is shaped the way it is, a complete file-by-file deep dive into the actual module, the infrastructure-as-code patterns it applies, and interview-prep Q&A — written as a standalone reference for a reader new to both Terraform and AWS.

Part 0 — Terraform / HCL, from zero

This is a different domain from the connector-layer docs' Python primers — Terraform's config language, HCL (HashiCorp Configuration Language), is declarative, not imperative: you describe the infrastructure you want to exist, not the steps to create it. No prior Terraform or AWS knowledge assumed.

0.1 The core idea: desired state, not a script

A Terraform `.tf` file is a description of what should exist — "there should be an ECR repository named `apm-connectors`," not "run this command to create one." Terraform's job is to compare that description against what actually exists in AWS right now (tracked in a **state file**, `terraform.tfstate`) and figure out, itself, what to create/change/destroy to close the gap. Run the same config twice with nothing changed, and the second run does nothing — this property is called **idempotence** (the connector-layer doc's §1.4 used the same word for a different context: same idea, "doing it again changes nothing").

0.2 The four commands you actually run

Command	What it does
<code>terraform init</code>	Downloads the providers a config needs (here: <code>hashicorp/aws</code> , <code>hashicorp/null</code>) -- run once per checkout, or after adding a provider.
<code>terraform plan</code>	Shows what apply would change, without changing anything -- a dry run in the same spirit as the connector layer's <code>dry_run</code> (that doc, §1.3).
<code>terraform apply</code>	Reconciles real AWS resources to match the config -- creates/updates/destroys whatever plan identified, after you confirm.
<code>terraform destroy</code>	Tears down every resource this config created. Used here to stop paying for the ALB/Fargate task when done (docs/deployment.md's 'Known limitations').

0.3 Blocks: the five shapes you'll see in this module

```

provider "aws" { region = var.aws_region }           # configures a plugin

resource "aws_ecr_repository" "this" {              # a thing Terraform creates/owns
  name = var.app_name
}

data "aws_vpc" "default" { default = true }          # reads something that already exists

variable "app_name" {                               # an input, filled in by terraform.tfvars
  type    = string
  default = "apm-connectors"
}

output "service_url" {                              # a value printed after apply
  value = "http://${aws_lb.this.dns_name}"
}

```

resource is the one you'll see most — each one is a real AWS object Terraform will create, and will destroy if removed from the config or on `terraform destroy`. Its two quoted labels are the resource *type* (`aws_ecr_repository` — which AWS thing) and a local *name* (`this` — how *this config* refers to it; not an AWS name). `data` looks almost identical but never creates or destroys anything — it just reads something that already exists (here, the account's default VPC) so other resources can reference it. `variable` and `output` are the module's inputs and outputs — the FastAPI doc's request/response shapes, for infrastructure.

0.4 References build an implicit dependency graph

```

resource "aws_ecr_repository" "this" {
  name = var.app_name
}

resource "aws_ecs_task_definition" "this" {
  # references the ECR repo's computed URL -- a value AWS assigns,
  # not something the config writer picks
  container_definitions = jsonencode([
    { image = "${aws_ecr_repository.this.repository_url}:latest" }
  ])
}

```

`aws_ecr_repository.this.repository_url` — `<type>.<name>.<attribute>` — reads an attribute off another resource, including ones AWS itself computes (a repository's URL isn't known until AWS creates it). Writing this reference is also how you tell Terraform *the task definition depends on the ECR repo existing first* — you never write "create A before B" explicitly; Terraform derives the entire creation order from which resources reference which. This is the same declarative-over-imperative idea as §0.1, one level more concrete.

0.5 String interpolation and heredocs

```

name = "${var.app_name}-alb"           # interpolation: embed an expression in a string

command = <<-EOT                       # heredoc: a multi-line string, indentation-stripped
  set -euo pipefail
  docker buildx build --platform linux/amd64 \
    -t ${aws_ecr_repository.this.repository_url}:${var.image_tag} --push .
EOT

```

`${...}` inside a string embeds any expression's value — variables, resource attributes, function calls. A `<<-EOT ... EOT` heredoc is how a multi-line value (here, an entire bash script) gets written without escaping every newline — the module uses this for the Docker build/push script (§2.2) and the force-redeploy script (§2.8), both handed to `local-exec` (§0.7).

0.6 count — conditional and repeated resources

```
resource "aws_ssm_parameter" "database_url" {
  count = var.database_url != "" ? 1 : 0    # 1 if set, 0 (i.e. don't create it) if not
  name  = "${var.app_name}/DATABASE_URL"
  value = var.database_url
}

# elsewhere, referencing a count-based resource needs an index:
aws_ssm_parameter.database_url[0].arn
```

`count` on a resource block turns it into a list of 0 or more instances — set to a ternary like condition `? 1 : 0`, this is Terraform's way of saying "only create this resource if...", since HCL has no plain `if` statement. This module uses exactly this for `google_token_json` and `database_url` (§2.6) — both optional secrets that, unlike the three always-created ones, shouldn't exist as an SSM parameter at all when unset. Referencing a count-based resource elsewhere always needs an index, even when there's at most one — `[0]`, guarded by the same condition.

0.7 for expressions, jsonencode, and functions

```
locals {
  source_hash = sha1(join("", [
    for f in fileset(local.repo_root, "src/**/*") : filesha1("${local.repo_root}/${f}")
  ]))
}

container_definitions = jsonencode([
  { name = var.app_name, image = "..." }
])
```

`[for f in ... : ...]` is a **for expression** — the same list-comprehension shape the connector-layer doc's §0.6 covered in Python (`[expr for item in iterable]`), here building a list of file hashes instead of a list of dict values. `fileset/filesha1/sha1/join` are Terraform's built-in functions (there's no way to define your own) — this specific chain is how `local.source_hash` (§2.1, §2.8) is computed: hash every tracked source file, then hash the list of hashes into one value. `jsonencode(...)` turns an HCL value (a list of objects here) into a JSON string at plan time — ECS's `container_definitions` argument is literally a JSON string, not a native Terraform type, so this is how HCL data becomes it.

0.8 The escape hatch: provisioners, null_resource, and triggers

Terraform models AWS resources declaratively — but building a Docker image and pushing it isn't an AWS resource at all; it's an imperative action with side effects Terraform has no built-in concept of. `null_resource` plus a `local-exec` provisioner is the escape hatch for exactly this:

```
resource "null_resource" "docker_build_push" {
  triggers = {
    source_hash = local.source_hash    # re-run ONLY when this value changes between applies
  }
  provisioner "local-exec" {
    interpreter = ["bash", "-c"]
    command     = "docker buildx build ... --push ."
  }
}
```

A `null_resource` manages nothing in AWS by itself — it exists purely to attach a provisioner (an arbitrary local command) to Terraform's apply lifecycle. Its `triggers` map is the only thing that decides whether it re-runs: Terraform compares each trigger value to what it was on the last apply, and re-runs the provisioner only if something in that map actually changed. Without a trigger, `local-exec` would only run once, ever, on first creation — this module uses `triggers` deliberately, twice (§2.2, §2.8), to make an otherwise

invisible-to-Terraform change (new source code; a changed secret value) actually cause something to happen on the next apply.

Part 1 — Why this deployment is shaped this way, from zero

1.1 The problem: run a FastAPI container somewhere public

The FastAPI doc covered uvicorn serving the app on `127.0.0.1:8000` — fine on a laptop, useless to anyone else. Deploying means: package the app so it runs the same way anywhere (a **container** — the repo-root Dockerfile builds one, installing this package and running the exact same uvicorn command inside it), run that container on a machine reachable from the public internet, and keep it running (restart it if it crashes, replace it on a new deploy) without babysitting it by hand.

1.2 ECS Fargate: containers without managing servers

ECS (Elastic Container Service) is AWS's service for running containers; **Fargate** is one way to run them — "serverless" in the sense that you never provision or patch an EC2 virtual machine yourself. You describe a **task definition** (what image, how much CPU/memory, what ports) and AWS finds the underlying capacity to run it. The module's comment notes this replaced an earlier design on **App Runner** (an even higher-level "just run this container" service) after AWS stopped accepting new App Runner customers — Fargate is one layer more manual (you define the load balancer and networking yourself, §1.3) but has no such sunset risk, being a mature, widely-used AWS service.

1.3 The network model: default VPC, a public ALB, and a private task

A few AWS networking primitives, each doing one job here:

Primitive	What it is	Its job in this module
VPC	An isolated virtual network in your AWS account	Every account gets a default one for free -- this module reuses it (§1.3 below) instead of building a new one.
Subnet	A slice of a VPC's IP range, in one availability zone	The default VPC's subnets are public (route to an Internet Gateway) -- used as-is for both the ALB and the task.
Security group	A stateful firewall attached to a resource	Two are defined (§2.3): one lets the public reach the ALB; the other lets only the ALB reach the task.
ALB	Application Load Balancer -- an HTTP(S) reverse proxy AWS runs for you	The one fixed public entry point; health-checks /health and forwards to the running task.

The deliberate choice worth understanding is **reusing the account's default VPC** rather than building a dedicated one. A from-scratch VPC for production usually adds private subnets plus a **NAT Gateway** so a task can reach the internet (to pull its image, call Gmail/Salesforce/Jira) without a public IP of its own — safer, but a NAT Gateway alone costs roughly as much per month as the ALB. This module instead gives the task a public IP directly (`assign_public_ip = true`, §2.7) in the default VPC's already-public subnets, and relies entirely on the **security group** — not network topology — to stop the public from reaching it: only the ALB's security group may reach port 8000 (§2.3). A public IP existing is not the same as being reachable; the security group is the actual gate.

1.4 Secrets: SSM Parameter Store instead of plain environment variables

A plain environment variable on a task definition is visible to anyone who can read that task definition via the AWS console or CLI — fine for a non-secret value like `JIRA_BASE_URL`, wrong for a client secret or API token. **SSM Parameter Store's** `SecureString` type encrypts the value at rest (via AWS KMS) and requires an explicit IAM permission (`ssm:GetParameters`, §2.5) to read it back — a real access-control boundary, not just a naming convention. ECS's task definition `secrets` field (as opposed to its `environment` field, §2.7) tells the execution role to resolve each one from SSM at container startup and inject it as a real env var inside the running container — the app itself (`config.load_settings`) never knows or cares that some of its env vars came from SSM and others were set directly.

1.5 The problem Terraform can't see on its own, and the fix

Terraform's whole reconciliation model (§0.1) depends on being able to tell when something changed. Two real things here change without Terraform noticing: pushing a new Docker image to the *same* tag (`:latest`) leaves the task definition's image string byte-for-byte identical; and an SSM `SecureString`'s *value* changing in place leaves its ARN — the only thing the task definition's `secrets` block references — unchanged too, and ECS only resolves secrets once, at container startup, so a running task wouldn't even notice if it did look. The content-hash triggers pattern (§0.8) is the fix for both: compute a hash of the actual thing that matters (source files; the secret values themselves) and use *that* as the signal a `null_resource` watches, sidestepping Terraform's normal attribute-diffing entirely.

Part 2 — This repo's module, file by file

File	Role
<code>infra/aws/ecs-fargate/main.tf</code>	Every resource -- ECR, Docker build/push, security groups, ALB, IAM, SSM secrets, ECS cluster/task/service, the force-redeploy trigger (419 lines).
<code>infra/aws/ecs-fargate/variables.tf</code>	Every input this module accepts -- deploy-shape knobs (region, cpu/memory) plus every connector's optional config (126 lines).
<code>infra/aws/ecs-fargate/outputs.tf</code>	The four values printed after apply -- <code>service_url</code> is the one you actually use.
<code>infra/aws/ecs-fargate/terraform.tfvars.example</code>	Copy to <code>terraform.tfvars</code> (gitignored) and fill in; the template for real, local, un-committed values.

2.1 Provider block and the source-hash locals

```
terraform {
  required_version = ">= 1.5"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" { region = var.aws_region }

locals {
  repo_root = abspath("${path.module}/../../..")
  source_hash = sha1(join("", [
    for f in fileset(local.repo_root, "src/**/*") : filesha1("${local.repo_root}/${f}")
  ]))
  dockerfile_hash = filesha1("${local.repo_root}/Dockerfile")
}
```

~> 5.0 pins the AWS provider to the 5.x line — allows patch/minor upgrades, blocks an accidental breaking major-version jump. `path.module` is a built-in reference to the directory this `.tf` file lives in, letting `local.repo_root` walk back up to the actual repository root regardless of where `terraform apply` is run from. `source_hash` and `dockerfile_hash` are the values §1.5's fix depends on — computed once here, referenced by two different `null_resources` later (§2.2, §2.8).

2.2 ECR + the Docker build/push escape hatch

```
resource "aws_ecr_repository" "this" {
  name          = var.app_name
  force_delete = true
}

resource "null_resource" "docker_build_push" {
  depends_on = [aws_ecr_repository.this]
  triggers = {
    source_hash      = local.source_hash
    dockerfile_hash = local.dockerfile_hash
    image_tag        = var.image_tag
  }
  provisioner "local-exec" {
    interpreter = ["bash", "-c"]
    working_dir = local.repo_root
    command = replace(<<-EOT
      aws ecr get-login-password --region ${var.aws_region} | docker login ...
      docker buildx build --platform linux/amd64 \
        -t ${aws_ecr_repository.this.repository_url}:${var.image_tag} --push .
    EOT
    , "\r\n", "\n")
  }
}
```

`force_delete = true` lets `terraform destroy` remove the ECR repository even with images still in it, rather than failing and requiring a manual cleanup first. The `replace(..., "\r\n", "\n")` wrapping the heredoc is a real, live-verified fix, not defensive boilerplate: on Windows with Git's default `core.autocrlf=true`, a checkout rewrites this file's line endings to CRLF, which corrupts the script by the time `bash` tries to run it — forcing LF here makes the script correct regardless of how the repo was checked out. `--platform linux/amd64` is there because Fargate only runs x86_64 — without it, running `apply` from an Apple Silicon or Windows-on-ARM machine would build an arm64 image that simply fails to start on Fargate.

2.3 Security groups — two tiers, one direction of trust

```
resource "aws_security_group" "alb" {
  ingress { from_port = 80, to_port = 80, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"] }
  egress { from_port = 0, to_port = 0, protocol = "-1", cidr_blocks = ["0.0.0.0/0"] }
}

resource "aws_security_group" "service" {
  ingress {
    from_port = 8000
    to_port   = 8000
    protocol = "tcp"
    security_groups = [aws_security_group.alb.id] # NOT a cidr_blocks -- only the ALB, ever
  }
  egress { from_port = 0, to_port = 0, protocol = "-1", cidr_blocks = ["0.0.0.0/0"] }
}
```

The service security group's `ingress` references *another security group* (`aws_security_group.alb.id`) instead of a CIDR block — this is a live, dynamic membership check ("traffic from something currently in the ALB

security group"), not a fixed IP range, and it's what actually enforces §1.3's claim that the task's public IP grants no direct access: nothing except the ALB's security group can reach port 8000, full stop, regardless of source IP. Both groups' unrestricted egress is deliberately broad — the service needs outbound access to ECR, CloudWatch, SSM, and every external connector API (Gmail, Salesforce, Jira, and optionally Postgres), which is an open-ended set of destinations not worth enumerating for an MVP deploy.

2.4 The load balancer, target group, and listener

```
resource "aws_lb" "this" {
  internal            = false
  load_balancer_type = "application"
  security_groups     = [aws_security_group.alb.id]
  subnets            = data.aws_subnets.default.ids
}

resource "aws_lb_target_group" "this" {
  port          = 8000
  target_type   = "ip"          # Fargate tasks are targeted by IP, not by EC2 instance id
  health_check {
    path = "/health", matcher = "200", interval = 15, timeout = 5
    healthy_threshold = 2, unhealthy_threshold = 5
  }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.this.arn
  port              = 80
  default_action { type = "forward", target_group_arn = aws_lb_target_group.this.arn }
}
```

Three resources, one pipeline: the `aws_lb` is the load balancer itself (the thing with a public DNS name — §2.9's `service_url` output); the listener says "accept HTTP on port 80"; the `target_group` is who traffic actually forwards to, and owns the health check against `/health` that decides whether a task is considered healthy enough to receive traffic at all. `target_type = "ip"` matters specifically because Fargate has no persistent EC2 instance to target — each task gets an ephemeral IP, and the target group tracks tasks by that IP instead.

2.5 IAM — an execution role, scoped narrowly

```
resource "aws_iam_role" "execution" {
  assume_role_policy = data.aws_iam_policy_document.execution_trust.json #
    ecs-tasks.amazonaws.com
}

resource "aws_iam_role_policy_attachment" "execution_managed" {
  role          = aws_iam_role.execution.name
  policy_arn    = "arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy"
}

data "aws_iam_policy_document" "execution_ssm_read" {
  statement {
    actions = ["ssm:GetParameters"]
    resources = local.ssm_secret_arns # only THESE parameters, never ssm:* on everything
  }
}
```

This is an **execution role** — what ECS itself uses to pull the image, write logs, and resolve secrets, distinct from a **task role** the running application code would use to call AWS APIs on its own behalf (this app makes no AWS API calls itself, so it has none). The AWS-managed `AmazonECSTaskExecutionRolePolicy` covers image pulls and log writes broadly, but grants no SSM access at all — the custom `execution_ssm_read`

policy adds exactly one narrow permission (`ssm:GetParameters`) scoped to exactly the parameter ARNs this deployment actually created (`local.ssm_secret_arns`, §2.6), rather than a blanket `ssm:*` on `"*"` — a deliberate, minimal grant, not the path of least resistance.

2.6 SSM secrets — three always, two conditional

```
resource "aws_ssm_parameter" "google_client_secret" {
  name = "/${var.app_name}/GOOGLE_CLIENT_SECRET"
  type = "SecureString"
  value = var.google_client_secret != "" ? var.google_client_secret : "unset"
}
# ...salesforce_client_secret, jira_api_token follow the same always-created shape

resource "aws_ssm_parameter" "database_url" {
  count = var.database_url != "" ? 1 : 0 # conditional -- see §0.6
  name = "/${var.app_name}/DATABASE_URL"
  type = "SecureString"
  value = var.database_url
}
```

Two different treatments for two different kinds of "empty."

`google_client_secret/salesforce_client_secret/jira_api_token` are always created, falling back to the literal string `"unset"` when the variable is empty — harmless, because a real connector call would fail on an actually-invalid credential anyway, and simpler than making the parameter itself conditional.

`google_token_json` and `database_url` use `count` instead, because an `"unset"` placeholder would be actively wrong for them: an empty `GOOGLE_TOKEN_JSON` env var should mean "fall back to the interactive OAuth flow" (`google_auth.py`, connector-layer doc §2.7), not "here's a literal token string that happens to say 'unset'"; and an empty `DATABASE_URL` should mean "no Postgres configured, use the file-backed store" (`config.py`'s `load_settings`), not a connection string the app would actually try and fail to connect to.

2.7 The ECS task definition and service


```

resource "aws_ecs_task_definition" "this" {
  requires_compatibilities = ["FARGATE"]
  network_mode             = "awsvpc"
  execution_role_arn       = aws_iam_role.execution.arn

  container_definitions = jsonencode([
    {
      name       = var.app_name
      image      = "${aws_ecr_repository.this.repository_url}:${var.image_tag}"
      portMappings = [ { containerPort = 8000, protocol = "tcp" } ]
      environment = [
        { name = "GOOGLE_CLIENT_ID", value = var.google_client_id },
        { name = "JIRA_BASE_URL", value = var.jira_base_url },
        # ...every other non-secret connector value
      ]
      secrets = local.container_secrets # resolved from SSM at startup -- see §1.4, §2.6
    }
  ])
}

resource "aws_ecs_service" "this" {
  depends_on = [aws_lb_listener.http, null_resource.docker_build_push]
  launch_type = "FARGATE"
  network_configuration {
    subnets          = data.aws_subnets.default.ids
    security_groups   = [aws_security_group.service.id]
    assign_public_ip = true # so the task can reach ECR/the internet -- see §1.3
  }
  load_balancer { target_group_arn = aws_lb_target_group.this.arn, container_port = 8000 }
}

```

The environment vs. secrets split on the container definition is §1.4's security boundary made concrete: every value in `environment` is a plain string right there in the task definition; every value in `secrets` is a `valueFrom` ARN reference the execution role resolves at startup. The service's explicit `depends_on` matters because Terraform's automatic dependency graph (§0.4) can't see everything here — the ECS service needs the listener to already exist (so its target group has somewhere to receive traffic from) and needs the Docker image already pushed (§2.2) before it tries to start a task from it, and neither of those is expressed through an attribute reference the graph would pick up automatically.

2.8 force_new_deployment — closing the loop from §1.5

```

resource "null_resource" "force_new_deployment" {
  depends_on = [aws_ecs_service.this, null_resource.docker_build_push]
  triggers = {
    source_hash = local.source_hash
    secrets_hash = sha1(join("", [
      var.google_client_secret, var.google_token_json, var.salesforce_client_secret,
      var.jira_api_token, var.database_url,
    ]))
  }
  provisioner "local-exec" {
    command = "aws ecs update-service --force-new-deployment ..."
  }
}

```

This is where §1.5's problem actually gets closed: a new image at the same tag, or a changed secret *value*, is invisible to Terraform's normal change detection but very visible to this `null_resource`'s hashed triggers — when either changes, it re-runs `aws ecs update-service --force-new-deployment` directly via the AWS CLI, which tells ECS to stop the running task(s) and start fresh ones (re-pulling the image, re-resolving every secret). The result: a plain `terraform apply` after any source or secret change is enough on its own — no separate manual redeploy command, ever.

2.9 variables.tf, outputs.tf, and terraform.tfvars

Every connector-specific variable defaults to an empty string — the Terraform-level mirror of an unfilled local .env (FastAPI doc's config-loading section): leaving one unset doesn't break apply, it just means that connector's /tools/* routes 503 once deployed, exactly like running locally with no credentials configured. Real values go in terraform.tfvars — gitignored, never committed — copied from the checked-in terraform.tfvars.example template. The four output values (Soutputs.tf) are what apply prints at the end; service_url is the one actually used afterward — the public HTTP URL to point scripts/api_smoke_test.py or a real caller at.

Part 3 — Infrastructure-as-code patterns used here, named explicitly

Pattern	Where	What problem it actually solves here
Declarative desired-state reconciliation	Every resource block; the whole init/plan/apply cycle (§0.1)	Re-running apply with nothing changed does nothing -- infrastructure described once, safe to re-apply indefinitely.
Content-hash trigger	local.source_hash/dockerfile_hash/secrets_hash driving two null_resources (§2.2, §2.8)	Surfaces changes (new source code, a rotated secret) that Terraform's own attribute-diffing structurally can't see.
Escape hatch via null_resource + local-exec	Docker build/push (§2.2); the forced ECS redeploy (§2.8)	Lets an imperative action with no native Terraform resource type (running Docker, calling the AWS CLI directly) still live inside the declarative apply lifecycle.
Secrets via a dedicated store, not plain config	SSM SecureString + the secrets/valueFrom split on the container definition (§1.4, §2.6, §2.7)	Client secrets and API tokens are encrypted at rest and gated by an explicit IAM read permission -- a real boundary, not a convention.
Least-privilege IAM	execution_ssm_read -- ssm:GetParameters scoped to local.ssm_secret_arns only (§2.5)	The execution role can read exactly the parameters this deployment created, nothing else in the account's SSM store.
Defense in depth via security-group chaining	aws_security_group.service's ingress references aws_security_group.alb.id, not a CIDR (§2.3)	The task is reachable only from something currently in the ALB's security group -- a public IP existing grants no access on its own.
Conditional resource creation	count = var.database_url != "" ? 1 : 0 (§0.6, §2.6)	An optional secret (Postgres URL, a pre-consented OAuth token) is only created as an SSM parameter -- and only costs an IAM grant -- when actually configured.
Fail-open-to-safe defaults	Every connector variable.tf entry defaults to "" (§2.9)	A deployment with zero connectors configured still applies cleanly and passes its health check -- same MVP-friendly default as an unfilled local .env.

Part 4 — Interview-prep Q&A

Practice answering by pointing at the actual code, not reciting the definition.

Q: What does 'declarative' actually mean here, concretely?

A: You write what should exist (an ECR repo named X, a task with these env vars), not the steps to get there. Terraform compares that description against its state file, computes the gap against real AWS resources, and decides itself what to create/change/destroy. Run apply twice with nothing changed and the second run is a no-op -- that idempotence is the practical payoff (§0.1).

Q: Terraform doesn't run your Docker build itself -- how does this module get an image built and pushed as part of `terraform apply`?

A: `null_resource.docker_build_push` (§2.2) -- a resource with no AWS meaning of its own, used purely to attach a local-exec provisioner (an arbitrary bash script) to the apply lifecycle. Its triggers map (a hash of every tracked source file plus the Dockerfile) is what decides whether the provisioner actually re-runs on a given apply, rather than every single time.

Q: Two real changes here are invisible to Terraform's normal change detection. What are they, and how does this module handle both?

A: A new image pushed to the same :latest tag -- the task definition's image string is textually unchanged. And an SSM SecureString's value changing in place -- its ARN, the only thing the task definition's secrets block references, doesn't change either, and ECS only resolves secrets once at task startup. Both are handled the same way: `null_resource.force_new_deployment` (§2.8) hashes the actual source/secret values as its trigger, and calls `aws ecs update-service --force-new-deployment` directly whenever that hash changes -- so a plain `terraform apply` alone is always enough, no manual follow-up command.

Q: Why does this module reuse the account's default VPC instead of creating a dedicated one, and what's the actual security tradeoff?

A: A dedicated VPC for a task that needs outbound internet access (to pull its image, call external APIs) usually means private subnets plus a NAT Gateway, which costs roughly as much per month as the ALB itself -- not worth it for an MVP deploy. Instead the task runs in the default VPC's already-public subnets with its own public IP (`assign_public_ip = true`), and the actual access control is the security group: `aws_security_group.service` only allows inbound from `aws_security_group.alb.id`, not from any CIDR block, so having a public IP doesn't by itself grant reachability (§1.3, §2.3).

Q: Walk through why a secret goes into the SSM `secrets` block instead of the task definition's plain `environment` list.

A: environment values sit as plain text right in the task definition -- readable by anyone who can view it via the console or CLI. secrets instead references an SSM SecureString's ARN (`valueFrom`), which the execution role resolves at container startup only because it holds an explicit `ssm:GetParameters` grant scoped to exactly those parameter ARNs (§2.5) -- a real IAM-enforced boundary. `GOOGLE_CLIENT_SECRET`/`SALESFORCE_CLIENT_SECRET`/`JIRA_API_TOKEN`/`GOOGLE_TOKEN_JSON`/`DATABASE_URL` all go through secrets; `GOOGLE_CLIENT_ID`/`JIRA_BASE_URL`/etc. -- not sensitive on their own -- go through environment (§1.4, §2.7).

Q: Why do `google_client_secret`/`salesforce_client_secret`/`jira_api_token` always get an SSM parameter (defaulting to the literal string "unset"), while `google_token_json`/`database_url` use count to skip creating one entirely when empty?

A: Because an empty value means something different for each. An unconfigured client secret would fail a real API call regardless of what placeholder it holds, so "unset" is harmless filler. But an empty `GOOGLE_TOKEN_JSON` is supposed to mean 'fall back to the interactive OAuth flow', and an empty `DATABASE_URL` is supposed to mean 'use the file-backed store' -- in both cases a literal 'unset' string would be actively wrong, read as a real (if broken) value instead of 'not configured'. `count = var.x != "" ? 1 : 0` (§0.6, §2.6) skips creating the parameter at all in that case, matching what the app already treats an absent env var as.

Vocabulary cheat-sheet

Term	Meaning here
State file	Terraform's record of what it last created, compared against your config on every plan/apply (§0.1).
Provider	A plugin (here, hashicorp/aws) that knows how to talk to one API/platform's resources.
Resource vs. data source	resource creates/owns a thing; data only reads something that already exists (§0.3).
Idempotent	Re-running the same operation again changes nothing further -- true of both terraform apply and, separately, a connector's read methods (connector-layer doc, §1.4).
null_resource + local-exec	The escape hatch for running an arbitrary local command (Docker build, an AWS CLI call) inside Terraform's apply lifecycle (§0.8).
Trigger	A value null_resource watches -- when it changes between applies, the attached provisioner re-runs (§0.8).
SSM Parameter Store / SecureString	AWS's key-value config/secret store; SecureString encrypts the value at rest and gates reads by IAM permission (§1.4).
ECS Fargate	AWS's serverless container-running service -- describe a task definition, AWS finds the capacity to run it, no EC2 instance to manage (§1.2).
ALB / target group / listener	The load-balancer pipeline: the listener accepts traffic on a port, the target group tracks and health-checks the actual backends, the ALB is the public-facing object with a DNS name (§2.4).
Execution role vs. task role	The execution role is what ECS uses to start the container (pull image, resolve secrets, write logs); a task role -- unused here -- is what the running application code itself would use to call AWS APIs (§2.5).

That's the whole deployment: one Terraform module that builds and pushes an image, stands up a public ALB in front of a private Fargate task in the account's default VPC, keeps every connector secret in SSM behind a narrowly-scoped IAM role, and closes the two gaps in Terraform's own change detection with a pair of content-hashed null_resources — the same propose-then-reconcile shape as everything else in this codebase, just applied to infrastructure instead of a Gmail send.